



리눅스 시스템 프로그래밍

5장 프로세스

목차

1. 프로세스 개념과 구조
2. 프로세스의 생성과 종료
3. 프로세스 교체

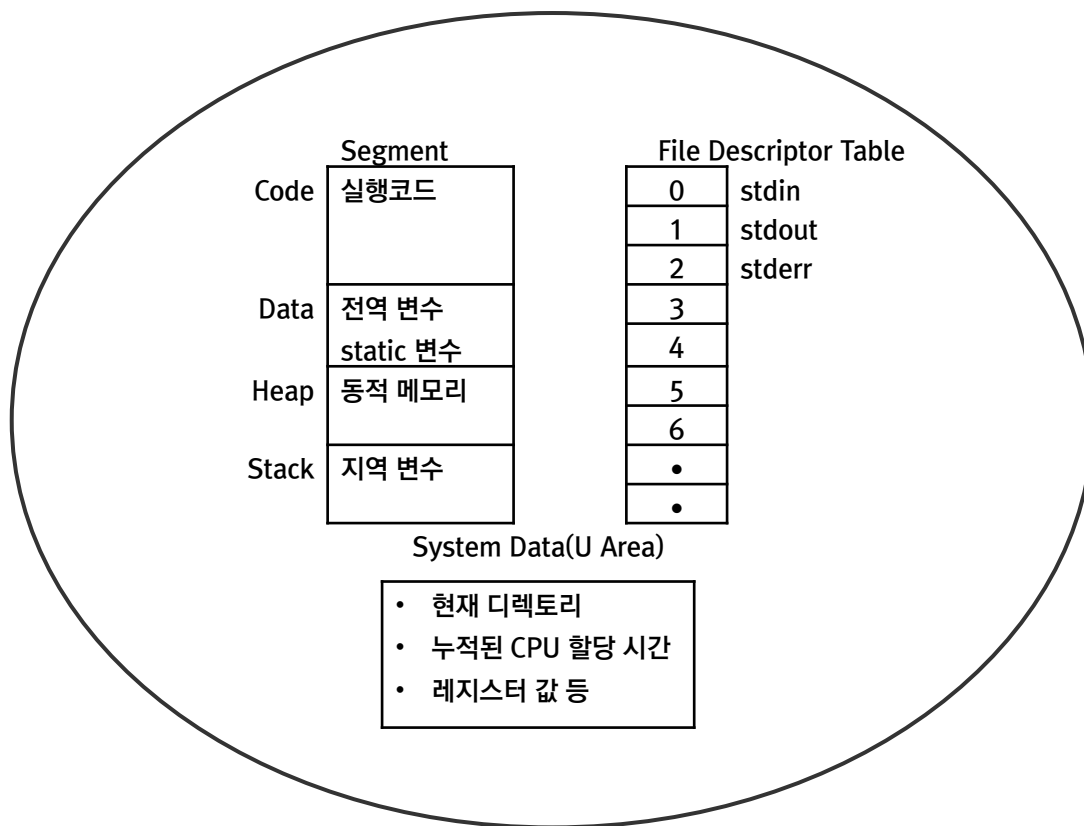
프로세스의 개념과 구조

1. 프로세스란?
2. 부모 프로세스와 자식 프로세스

프로세스란?

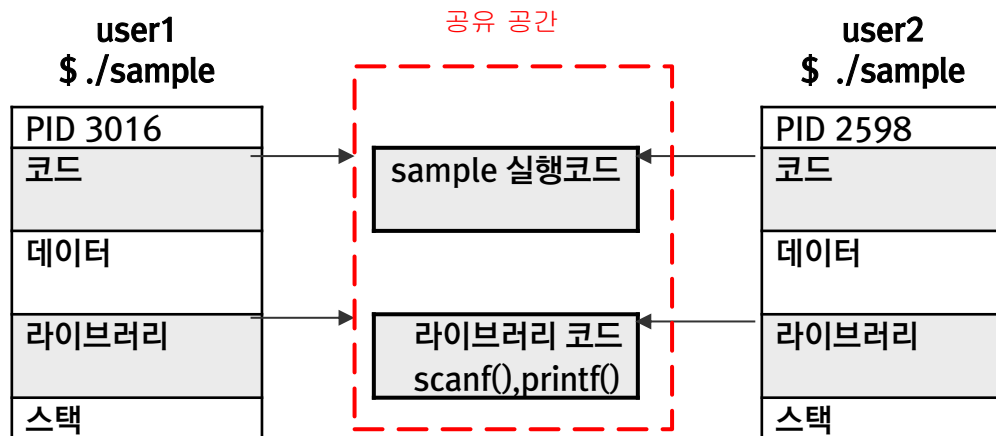
■ 프로세스 정의

리눅스와 같은 멀티 프로세싱 운영체제에서는 동시에 여러 프로그램을 실행할 수 있다. 이와 같이 실행중인 프로그램을 프로세스라고 한다. 즉 프로세스란 프로그램을 실행하는데 필요한 모든 환경을 통틀어 말하는 것으로 이 환경을 이미지 또는 컨텍스트(context)라고도 한다. 프로그램과 그 실행 환경, 즉 이미지를 구성하고 있는 요소들은 다음 그림과 같다



프로세스 구조

- 리눅스 운영체제는 여러 사용자가 동시에 액세스가 가능한 멀티 유저 시스템이므로 여러 프로그램이 동작되거나 같은 프로그램이 동시에 여러 프로세스로 수행될 수도 있다. 실제로 리눅스 시스템에서는 코드 영역과 라이브러리는 프로세스간에 공유하므로 메모리 내에 코드와 라이브러리는 하나만 존재한다. 예를 들어 두 명의 사용자가 동시에 동일한 sample 프로그램을 실행한다고 하자.



부모 프로세스와 자식 프로세스

- 일반적으로 리눅스에서 프로세스는 부모 프로세스에 의해 생성이 된다. 이렇게 만들어진 프로세스를 자식 프로세스라고 한다. 리눅스에 있는 모든 프로세스는 부모와 자식의 관계를 이루면서 수행되고 있다. `ps` 명령을 이용하여 프로세스의 관계를 확인해 보자.
- `ps` 명령의 수행결과를 살펴보면 프로세스 중 가장 조상이 되는 프로세스는 `systemd`로 PID가 1 임을 알 수 있다, 이 프로세스는 리눅스 커널이 초기화 과정을 수행한 후 마지막에 `systemd` 프로세스를 생성함에 따라 만들어진 프로세스로, 명령의 실행결과를 통해 시스템의 각종 대몬 프로세스를 생성하고 있다는 것을 알 수 있다. 이와 같이 자식 프로세스를 생성하여 프로그램을 수행하는 방법은 다음 절에서 학습할 `fork()`, `wait()`, `exec()` 등의 함수를 이용하여 구현된다.

부모 프로세스와 자식 프로세스

- 프로세스에 관련된 상세 정보에는 프로세스 고유의 식별자인 PID와 부모 프로세스의 PID, 우선순위, CPU 소모 시간, 프로세스 상태 등이 있으며 이 내용은 다음과 같이 'ps -l' 명령을 통해 확인할 수 있다.

```
user@ubuntu: ~/online_lsp/ch04
user@ubuntu:~/online_lsp/ch04$ ps -l
F S  UID    PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S   1000   5139  5130   0  80   0 -  6766 wait  pts/0      00:00:00 bash
0 R   1000  35587  5139   0  80   0 -  3553 -    pts/0      00:00:00 ps
user@ubuntu:~/online_lsp/ch04$
```

수행 결과 중 두 번째 필드의 S 항목은 프로세스의 상태를 나타내는 항목으로 그 의미는 다음과 같다 (man ps 참조)

프로세스 상태

■ 〈프로세스 상태 코드〉

상태 코드	의미
D	중단시킬 수 없는 대기 상태(usually IO)
R	준비 또는 실행 상태(on run queue)
S	대기 상태 (waiting for an event to complete)
T	멈춤 상태
W	페이징 중인 상태(not valid since the 2.6.xx kernel)
X	종료 상태
Z	좀비 상태

위에 보이는 프로세스 상태 코드는 리눅스에서 하나의 프로세스가 생성되어 소멸될 때까지의 상태를 코드로 표현한 것으로 시스템 프로그램 개발시 프로세스 상태는 중요한 변수가 될 수 있으므로 각 상태가 어떤 의미인지 잘 알아야 한다. 다음 절에서 이에 대해 잘 알아보자.

프로세스의 생성과 종료

1. 프로세스 생성
2. 프로세스 종료

프로세스 생성

- 리눅스에서 프로세스를 새로 생성하는 방법은 `system()` 이나 `fork()` 함수를 이용하는 방법이다. 이 두 함수에 대하여 알아보자.

- `system()` 함수

`system()` 라이브러리 함수를 이용하면 프로그램 수행도중 새로운 명령이나 프로그램을 수행할 수 있다. 이 함수의 원형은 다음과 같다,

```
#include <stdlib.h>
int system(const char *string);
```

- 이 함수를 수행하면 `string`에 명시한 명령이나 프로그램을 수행하고 종료를 기다린다. 마치 셸에서 명령을 입력하여 수행하는 것과 동일하다. `system()` 함수의 리턴 값은 `string`에 표현된 명령의 수행 결과를 의미한다. 0이 반환되면 정상적으로 수행되었음을, 0 아닌 값이 반환되면 정상적으로 수행되지 못했음을 의미한다

ex01.c 를 이용하여 system() 함수의 사용법을 익혀보자

소스코드 (ex01.c)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(void)
6  {
7      char cmd_buf[256];
8
9      printf("----- system() start ----- \n\n");
10     strcpy(cmd_buf, "sleep 3");
11     system(cmd_buf);
12     strcpy(cmd_buf, "ls");
13     system(cmd_buf);
14     printf("\n----- system() end ----- \n");
15     return 0;
16 }
```

프로세스 생성

■ fork() 함수

- 프로세스를 여러 개 동시에 수행하면 여러 가지 작업을 한번에 처리할 수 있다. 리눅스에서 프로세스를 생성하는 방법은 셸에서 프로그램을 직접 수행시키거나 fork() 라는 시스템 콜을 이용하는 것이다. fork() 함수의 원형은 다음과 같다.

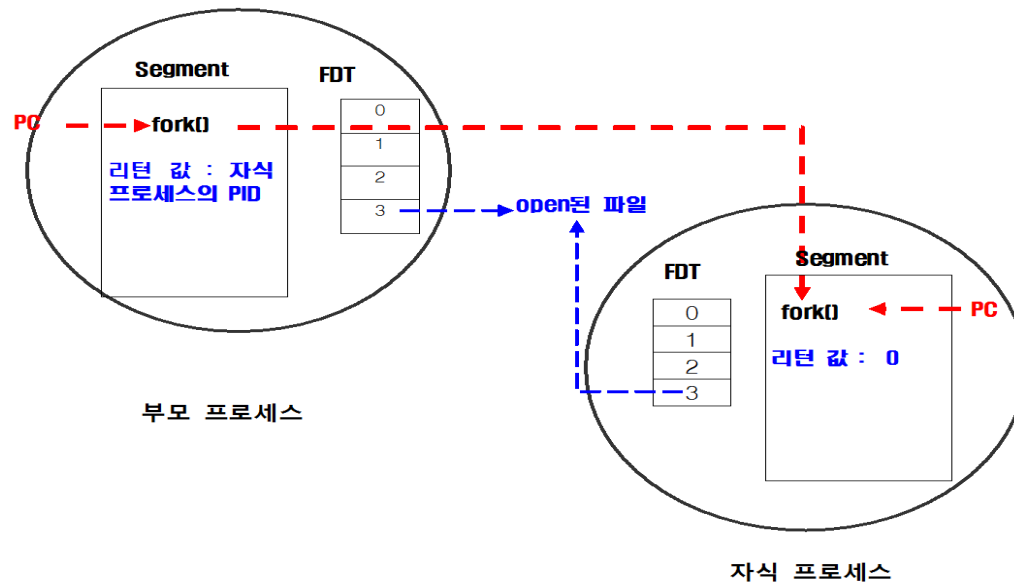
```
#include <unistd.h>
```

```
pid_t fork(void);
```

- fork() 함수를 수행하면 리눅스 커널에서는 이 함수를 호출한 프로세스와 동일하게 구성된 새로운 프로세스를 생성한다. 이때 함수를 호출한 쪽을 부모 프로세스라고 하고 새로 생성된 쪽을 자식 프로세스라고 한다.

프로세스 생성

- 자식 프로세스는 부모 프로세스와 같은 공간의 코드를 수행하지만 자기자신만의 데이터 공간 및 파일 디스크립터 등의 실행 환경을 구성한다. 새로 생성된 자식 프로세스는 고유한 PID(Process ID)를 갖게 되며, PPID에는 부모 프로세스의 PID를 간직하게 된다. 이외에 부모 프로세스로부터 시그널 등도 상속을 받는다. 그림을 통해 살펴보면 다음과 같다.



프로세스 생성

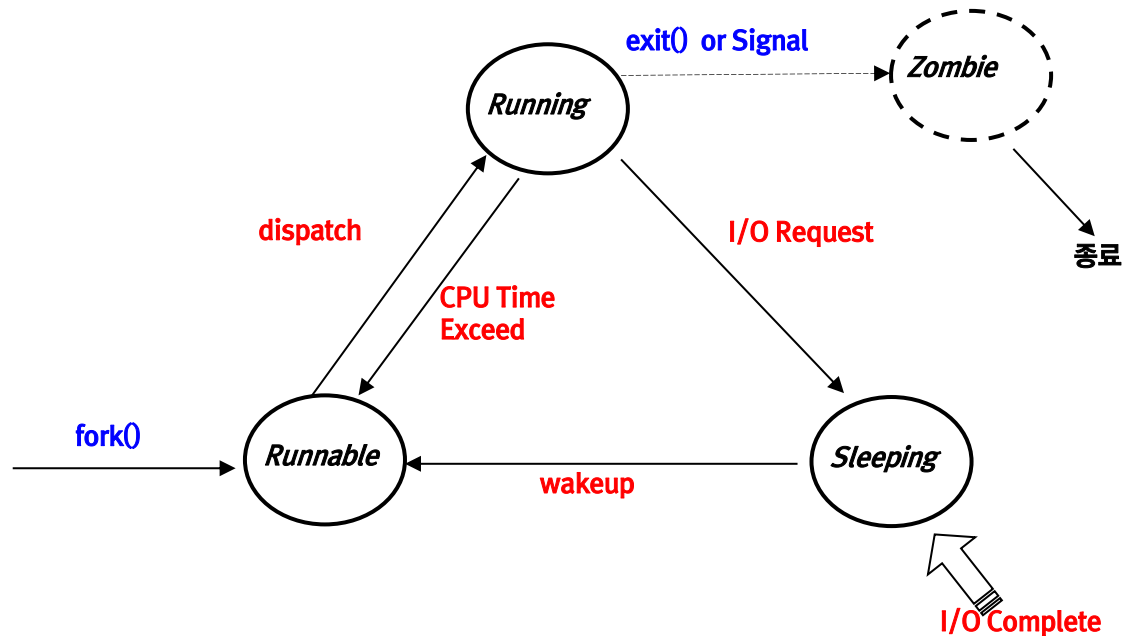
- `fork()` 함수가 실패하는 경우에는 `-1` 이 반환된다. 실패의 원인은 메모리 부족하거나 자식 프로세스의 개수의 제한에 걸리기 때문이다. 전자의 경우에는 `errno` 변수에 `ENOMEM`, 후자의 경우에는 `EAGAIN` 이 설정된다.
- `fork()` 함수를 이용하여 자식 프로세스를 생성하는 프로그램을 만들어보자 .

소스코드 (ex02.c)

```
...  
  
int main(void)  
{  
    pid_t pid;  
  
    switch (pid=fork())  
    {  
        case -1:  
            perror("fork");  
            break;  
        case 0: // child process  
            printf("CHILD PROCESS : %d\n", getpid());  
            sleep(3);  
            system("echo CHILD");  
            system("ps -l");  
            exit(0);  
            break;  
        default:  
            printf("PARENT PROCESS : %d\n", getpid());  
            printf("Return Value of fork : %d\n\n", pid);  
            sleep(7);  
            system("echo PARENT");  
            system("ps -l");  
            break;  
    }  
    return 0;  
}
```

프로세스 생성

- 앞서 살펴본 ex02.c 의 수행 결과에서 자식 프로세스가 종료되지 않고 Zombie 상태에 머문 이유는 무엇일까? Zombie 상태란 무엇을 의미하는지 알아보기에 앞서 우선 다음 그림을 통해 프로세스의 상태에 대하여 알아보자.



프로세스의 상태 천이

프로세스 생성

- 앞의 그림에서 프로세스 종료시 자식 프로세스가 보낸 시그널을 부모 프로세스가 받지 못하게 되면 자식 프로세스는 좀비 상태에 계속 머무르게 된다. 좀비 상태에 있는 프로세스 자체가 문제를 일으키지는 않지만 이 상태에 있는 프로세스가 점점 늘어나게 되면 더 이상 프로세스를 생성할 수 없는 경우가 발생할 수 있다. 리눅스에는 프로세스의 최대 개수가 설정되어 있기 때문이다. 다음 과 같은 명령을 통해 확인해보자.

```
$ cat /proc/sys/kernel/pid_max
```

- 현재 설정되어있는 PID의 최대값, 그 이상의 프로세스는 동시에 수행할 수 없다. 따라서 좀비 프로세스가 점점 늘어나게 되어 프로세스 전체 개수가 최대치에 이르게 되면 더 이상 작업을 수행할 수 없게 된다.
- 이렇게 리눅스 시스템을 다운 시킬 수도 있는 좀비 상태의 프로세스를 해결하기 위해 부모 프로세스가 마무리 해야 하는 것은 무엇일까?

프로세스 종료

- 리눅스에서 프로세스 종료에 관련된 함수는 wait(), waitpid() 및 exit() 등이다. 각 함수에 대하여 살펴보자.
- wait() 함수

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *stat_loc);
```

부모 프로세스가 자식 프로세스를 생성하면 거기에서 책임이 끝나는 것이 아니라 자식 프로세스의 종료까지도 책임져야 한다. 이 역할을 수행하는 함수가 wait() 함수다.

wait() 함수를 호출하면 자식 프로세스가 시그널을 보낼 때까지 부모 프로세스의 수행을 중지시킨다. 자식 프로세스로부터 시그널이 도착하면 함수를 빠져 나오면서 시그널을 보낸 자식 프로세스의 PID 를 반환해준다. 만약 자식 프로세스가 존재하지 않는 경우에는 -1 을 반환하고 errno 변수에 ECHILD를 설정한다. 부모 프로세스에서 자식 프로세스의 종료 원인을 좀더 자세히 알고 싶은 경우에는 인수로 상태 정보를 읽어올 수 있는 변수를 넘겨주어 종료 상태를 읽어올 수 있다.

프로세스 종료

- 종료 상태에 대한 정보는 `sys/wait.h` 에 정의되어있는 매크로를 이용하여 쉽게 해석할 수 있다. 다음은 종료 상태 관련 매크로들 정리한 표다

매크로	의미
WIFEXITED(stat_loc)	자식 프로세스가 <code>exit()</code> 함수로 종료한 경우 참
WEXITSTATUS(stat_loc)	자식 프로세스의 종료 코드 반환(단, WIFEXITED() 가 참일 때)
WIFSIGNALED(stat_loc)	자식 프로세스가 시그널에 의해 종료된 경우 참
WTERMSIG(stat_loc)	자식 프로세스가 SIGTERM 시그널에 의해 종료된 경우 참
WIFSTOPPED(stat_loc)	자식 프로세스가 실행을 멈춘 경우 참
WSTOPSIG(stat_loc)	자식 프로세스가 SIGSTOP 시그널에 의해 멈춘 경우 참

소스코드 (ex03.c)

```
...  
  
switch (pid=fork())  
{  
    case -1:  
        perror("fork");  
        break;  
    case 0: // child process  
        printf("CHILD PROCESS : %d\n", getpid());  
        sleep(3);  
        system("echo CHILD");  
        system("ps -l");  
        exit(0);  
        break;  
    default:  
        printf("PARENT PROCESS : %d\n", getpid());  
        printf("Return Value of fork : %d\n\n", pid);  
        wait(&status);  
        if(WIFEXITED(status)) {  
            printf("Child Process is deaded by exit0 \n");  
        } else if(WIFSIGNALED(status)) {  
            printf("Child Process is deaded by Signal\n");  
        }  
        system("echo PARENT");  
        system("ps -l");  
        break;  
}  
return 0;  
}
```

프로세스 종료

■ waitpid() 함수

- 앞서 살펴본 wait() 함수는 자식 프로세스의 종료를 기다렸다가 종료 신호가 오면 자식 프로세스를 삭제한다. 이때 자식 프로세스로부터 신호가 도착하지 않으면 부모 프로세스는 수행을 멈추고 기다린다. 만약 자식 프로세스로부터 신호가 오기 전에 부모 프로세스에서 수행해야 할 내용이 있다면 어떻게 해야 할까? 이런 경우에는 wait() 함수 대신 waitpid() 함수를 이용할 수 있다.

```
#include <sys/types.h>
#include <sys/wait.h>
```

```
pid_t waitpid(pid_t pid, int *stat_loc, int option);
```

pid

- > 0 지정한 자식 프로세스의 종료 신호를 기다림
- 0 같은 그룹에 있는 자식 프로세스의 종료 신호를 기다림
- 1 임의의 자식 프로세스의 종료를 기다림

option

- 0 자식 프로세스의 종료를 기다림
- WNOHANG 자식 프로세스의 종료를 기다리지 않고 즉시 리턴

- waitpid() 함수에서는 wait() 함수에서 설정할 수 없었던 특정 프로세스에 대한 설정이나 자식 프로세스로부터 시그널이 도착될 때까지 동작을 조정할 수 있다.

프로세스 종료

- 사용 예

```
waitpid(1023, NULL, WNOHANG);
```

이 함수를 수행하면 자식 프로세스의 PID가 1023인 프로세스가 종료 신호를 보내지 않은 경우 0을 반환하고, 종료하였다면 자식 프로세스의 PID 를 반환하게 된다. 만약 오류가 발생되었다면 -1을 반환하면서 errno 변수에 다음과 같은 값이 설정된다.

ECHILD	자식 프로세스가 존재하지않는 경우
EINTR	시그널에 의해 wait() 함수가 중단된 경우
EINVAL	옵션 부분이 유효하지 않은 경우

ex04.c 소스를 확인하고 실행해봅시다.

```
switch(c_pid1=fork()) {
    case -1:
        perror("fork1");
        exit(1);
        break;
    case 0:
        printf("%d : long time child process\n", getpid());
        sleep(10);
        exit(0);
        break;
    default:
        switch(c_pid2=fork()) {
            case -1:
                perror("fork2");
                exit(1);
                break;
            case 0:
                printf("%d : short time child process\n",getpid());
                sleep(1);
                exit(0);
                break;
        }
}
```

```
while(1) {
    while((ret_pid=waitpid(-1, NULL, WNOHANG))>0) {
        printf("\nreturn value of waitpid() : %d\n", ret_pid);
    }
    if(errno==ECHILD) {
        printf("no child\n");
        break;
    }
    if(i==100000) {
        putchar('.');
        fflush(stdout);
        i=0;
    }
    i++;
}
break;
}
```

프로세스 종료

■ `exit()` 함수

- 이 함수는 프로세스가 자신을 종료할 때 사용하는 함수로 원형은 다음과 같다.

```
#include <stdlib.h>

void exit(int status);
```

- `exit()` 함수의 인수 `status`는 `wait()`나 `waitpid()` 함수를 이용하여 자식 프로세스의 종료를 기다리는 부모 프로세스에 전달된다. 일반적으로 정상적인 종료일 때에는 `status`에 0을, 비정상적인 종료일 때에는 0~255 사이의 값을 전달한다. 이 함수를 호출하면 지금까지 사용하던 입출력 버퍼를 비우고, open 되어있는 파일 디스크립터를 모두 close하며 마지막에 부모 프로세스에게 SIGCHLD 시그널을 전송한다. 따라서 부모 프로세스에서는 `wait()` 또는 `waitpid()` 함수를 이용하여 이 시그널을 처리해주어야 한다.
- `ex05.c` 를 통해 `exit()` 함수의 종료 코드 값을 확인해보자

소스코드 (ex05.c)

```
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void)
{
    int status;

    switch(fork())
    {
        case -1:
            perror("fork");
            exit(1);
            break;
        case 0:
            sleep(3);
            exit(0);
            //exit(1);
            break;
        default:
            wait(&status);
            if(WIFEXITED(status)) {
                printf("exit code of child process : %d\n", WEXITSTATUS(status));
            }
    }
}
```

〈TIP〉 ZOMBIE 프로세스 찾아내기

- 현업에서 실행중인 프로그램 중 종종 좀비 프로세스가 발견되곤 한다. 좀비 프로세스를 찾아내는 방법은 무엇일까?
- 모범답안
 - 좀비 프로세스란 이미 프로세스는 종료되었지만 부모 프로세스에 의해 종료처리가 안된 상태를 의미한다 리눅스 시스템에는 동시에 수행가능한 프로세스의 개수가 정해져 있으므로 (/proc/sys/kernel/pid_max) 좀비 프로세스가 늘어나면 더 이상 프로세스의 생성이 불가능한 경우가 발생할 수 있다.이렇게 되면 시스템에 장애가 발생하게 된다. 따라서 사전에 좀비 프로세스가 있는지 찾아봐야 한다.
 - 좀비 프로세스를 검색하는 방법은 다음 명령에 의해 가능하다.

```
ps -el | awk '$2=="Z" { print $0 }'
```

여기에서 PPID 항목에 보이는 프로세스가 부모 프로세스로 문제의 원인이 되는 프로세스이다.

프로세스 교체

■ exec 함수

- 현재 수행중인 프로세스를 새로운 프로세스로 교체하는 경우 다음과 같은 exec() 함수들을 이용한다. 함수의 원형은 다음과 같다.

```
#include <unistd.h>
```

```
int execl(const char *path, const char *argv0, ....., (char *) 0);
```

```
int execlp(const char *file, const char *argv0, ....., (char *) 0);
```

```
int execlle(const char *path, const char *argv0, ....., (char *) 0, char *const envp[]);
```

```
int execv(const char *path, char *const argv[]);
```

```
int execvp(const char *file, char *const argv[]);
```

```
int execlve(const char *path, char *const argv[], char *const envp[]);
```

- exec() 함수들의 첫번째 인수는 실행 파일을 지정하고, 두 번째 인수부터는 그 실행 파일을 실행하는 인수들을 지정한다.

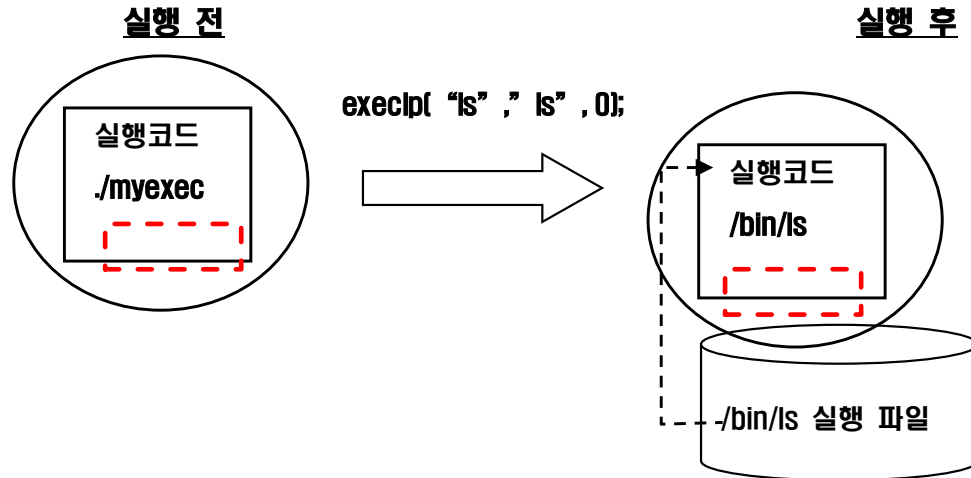
.

프로세스 교체

exec_	의미
l	list 함수로 인수들의 목록을 함수의 매개변수에 개별적으로 전달하여 실행하며, 인수들의 개수를 알 수 있을 때 사용한다 예) <code>execl("/bin/ls", "ls", "-l", (char *) 0);</code>
v	vector 함수로 인수의 목록을 argv 시작 주소를 이용하여 전체 전달한다. 예) <code>char *const ls_arg[] = { "ls", "-l", 0};</code> <code>execv("/bin/ls", ls_arg);</code>
p	셸의 환경변수 PATH에 정의된 경로를 이용하여 실행 파일을 찾아 실행한다. 따라서 첫 번째 인수에 경로 없이 실행 파일 명만 지정하면 된다. 예) <code>execlp("ls", "ls", "-l", (char *) 0);</code> <code>execvp("ls", ls_arg);</code>
e	실행 파일 수행 시 필요한 환경 변수 envp 로 전달하여 실행한다 예) <code>char *const envp[] = { "PATH=/bin:/usr/bin", "TERM=xterm", 0};</code> <code>execle("/bin/ls", "ls", "-l", (char *) 0, envp);</code> <code>execve("/bin/ls", ls_arg, envp);</code>

- 위와 같은 exec() 함수들은 현재 수행되던 프로세스를 새로운 프로세스로 교체하여 실행하게 해준다. 즉, 다음 그림과 같다.

프로세스 교체



- 여기서 주의 할 것은 프로세스의 구조 중 **세그먼트만 교체**된 다는 것이다. 파일 디스크립터 테이블(FDT)이나 시그널 핸들러와 같은 환경은 변경되지 않는 다는 점이다. 예를 들어 위 그림 에서처럼 실행 전 myexec 프로세스가 오픈 해놓은 파일이 있었다면 `execvp()` 함수 수행 후 ls 프로세스가 그 오픈 해놓은 파일을 이용할 수 있다는 의미이다.

ex06.c 소스를 확인하고 실행해봅시다.

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(void)
{
    pid_t pid;

    pid=fork();
    switch(pid) {
    case -1:
        perror("fork()");
        exit(-1);
        break;
    case 0:    /* child process */
        printf("CHILD Process\n");
        execlp("ls", "ls", "-l", (char *)0);
        perror("execlp");
        exit(0);
        break;
    default:  /* Parent Process */
        wait(NULL);
        printf("child is dead\n");
        break;
    }
    return 0;
}
```

〈실습〉

- 다음은 새로운 프로세스를 생성하여 자식 프로세스에서 'ps -l' 명령을 실행하는 프로그램을 완성하십시오.

소스코드 (exercise.c)

```
.....  
  
int main(void)  
{  
    pid_t pid;  
  
    pid= fork ();  
    switch(pid){  
    case -1:  
        perror("fork()");  
        exit(-1);  
        break;  
    case 0:    /* child process */  
  
        break;  
    default:  /* Parent Process */  
  
        break;  
    }  
    return 0;  
}
```



리눅스 시스템 프로그래밍

6.1장 시그널



목차

1. 시그널이란?
2. 시그널 핸들러
3. 시그널 전송

시그널이란

■ 시그널 종류와 사용

- 시그널이란 시스템에서 발생하는 사건(event)을 프로세스에게 알려주어 그에 따른 동작을 수행하도록 하는 메소드를 말한다. 여기서 사건이란 사용자가 발생하는 인터럽트나 부동 소수점 오류, 알람과 같은 것을 말한다. 시그널에 따라 기본적으로 수행하는 동작을 디폴트 핸들러가 설정되어 있어 그에 따른 동작을 처리하게 된다. 디폴트 핸들러의 종류는 다음과 같다.

핸들러 종류	동작
Term	프로세스 종료
Ign	시그널 무시
Core	Core 파일을 생성하며 프로세스 종료
Stop	프로세스 정지
NoCatch	사용자의 핸들러 설정 불가
NoIgn	시그널 무시 불가(블록킹 할 수 없음)

Core : 컴퓨팅에서의 **코어 덤프(core dump)**, 메모리 덤프(memory dump), 또는 시스템 덤프(system dump)^[1]는 보통 프로그램이 비정상적으로 종료(충돌)되는 때와 같은 특정 시점에 컴퓨터 프로그램에 기록된 작업 메모리로 구성된다.^[2] 실제로, 프로그램 카운터와 스택 포인터, 메모리 관리 정보, 기타 프로세서와 운영 체제 플래그 및 정보 등을 포함한 프로세서 레지스터같은 프로그램 상태의 중요 부분들이 대개 동시에 덤프 된다. 코어 덤프는 종종 컴퓨터 프로그램의 오류를 진단하고 디버깅하는 데 사용된다. <출처 : 위키백과사전>

- 시그널 별 디폴트 핸들러는 다음과 같은 명령을 통해 확인할 수 있다.

```
$ man 7 signal
```

시그널이란

- 사용자 또는 응용프로그램 생성 시그널 (signal.h)

시그널 명	시그널 번호	의미
SIGHUP	1	행업(hangup) 신호
SIGINT	2	터미널에서의 인터럽트 신호(Control C)
SIGQUIT	3	터미널에서의 종료 신호(Control \), core 파일 생성
SIGABRT	6	프로세스 중단
SIGKILL	9	가장 강력한 종료 신호
SIGUSR1	10	사용자 정의 신호
SIGUSR2	12	사용자 정의 시그널
SIGTERM	15	일반적인 종료 신호

- 표에 기술된 시그널의 디폴트 핸들러는 종료처리다. 즉, 이 시그널을 받은 프로세스는 즉시 종료하게 된다. 이 중 SIGINT, SIGQUIT는 터미널의 키보드로부터 발생하는 시그널로 현재 수행중인 포그라운드 프로세스에 전달된다. SIGINT 는 키보드에서 Control C 를 누른 경우에, SIGQUIT 는 Control \ 를 눌렀을 때 발생하는 시그널이다.

시그널이란

■ SIGINT, SIGQUIT 시그널 예

```
user@ubuntu: ~/Desktop
user@ubuntu:~/Desktop$ sleep 60
^C
user@ubuntu:~/Desktop$
user@ubuntu:~/Desktop$ sleep 60
^\\Quit (core dumped)
user@ubuntu:~/Desktop$ ls
core
user@ubuntu:~/Desktop$
```

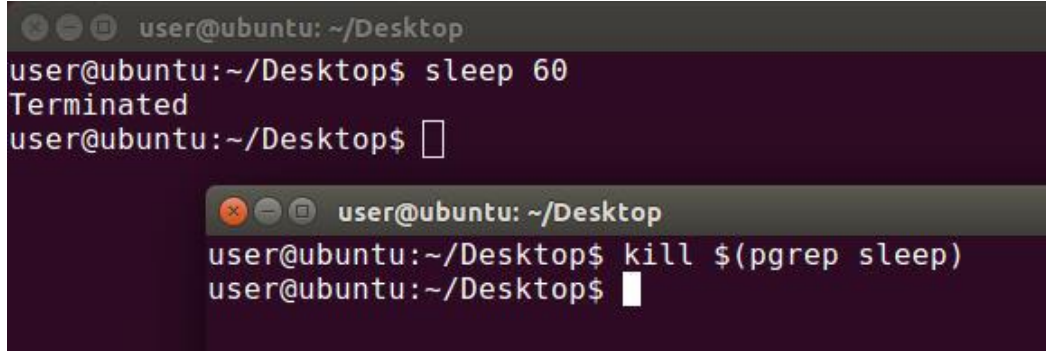
- 그림은 sleep 명령 수행시 키보드에서 Control C를 눌러 SIGINT 시그널을 발생한 경우와 Control \를 눌러 SIGQUIT 시그널을 발생한 경우의 화면이다. SIGINT가 발생한 경우는 sleep 프로세스가 그냥 종료되지만 SIGQUIT 시그널이 발생된 경우에는 core 파일이 생성되면서 프로세스가 종료되는 것을 알 수 있다.
- 프로세스에 시그널을 전송할 수 있는 또 다른 방법은 kill 명령을 이용하는 것으로 사용법은 다음과 같다.

\$ kill [-시그널번호] <pid>

명령을 수행하면 pid 프로세스에 시그널이 전송된다. 시그널 번호를 생략한 경우에는 기본적으로 SIGTERM 시그널이 전송된다.

시그널이란

■ kill 명령의 사용 예



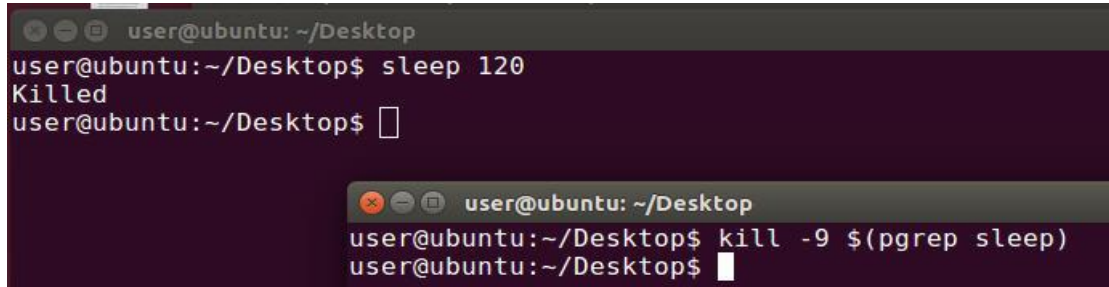
```
user@ubuntu: ~/Desktop
user@ubuntu:~/Desktop$ sleep 60
Terminated
user@ubuntu:~/Desktop$ █

user@ubuntu: ~/Desktop
user@ubuntu:~/Desktop$ kill $(pgrep sleep)
user@ubuntu:~/Desktop$ █
```

- 그림은 sleep 프로세스에 SIGTERM 시그널을 전송하여 강제로 종료시킨 예의 화면이다. 이 시그널을 받은 sleep 프로세스는 “Terminated” 메시지를 출력한 후 즉시 종료된다. 여기서 \$(pgrep sleep) 은 sleep 프로세스의 pid 를 검색하는 명령으로 이 명령의 수행결과를 직접 kill 명령의 인자로 전달하였다.
- SIGTERM 시그널을 수신한 프로세스는 이 시그널을 보낸 프로세스의 권한 등을 확인한 후 유효한 시그널인 경우 프로세스를 종료시킨다. 따라서 이 시그널로 종료되지 않는 프로세스도 있다. 그리고 이 시그널은 사용자가 핸들러를 변경하거나 무시하도록 설정할 수도 있다.

시그널이란

- kill 명령을 이용하여 SIGKILL 신호를 전송한 예



```
user@ubuntu: ~/Desktop
user@ubuntu:~/Desktop$ sleep 120
Killed
user@ubuntu:~/Desktop$ █

user@ubuntu: ~/Desktop
user@ubuntu:~/Desktop$ kill -9 $(pgrep sleep)
user@ubuntu:~/Desktop$ █
```

- 그림은 kill 명령을 이용하여 sleep 프로세스에게 SIGKILL 시그널을 전송한 화면이다. SIGKILL 시그널을 수신한 sleep 프로세스는 “Killed” 메시지를 출력하고 즉시 종료한다. SIGKILL 시그널을 수신하면 프로세스는 무조건 종료되며, 프로세스에서 이 시그널을 무시하거나 다른 핸들러를 등록할 수 없다. SIGKILL 과 같이 무시하거나 핸들러를 변경할 수 없는 시그널에는 SIGSTOP도 있다,
- 앞서 언급된 시그널 이외에도 다음과 같은 시그널들이 존재한다.

시그널이란

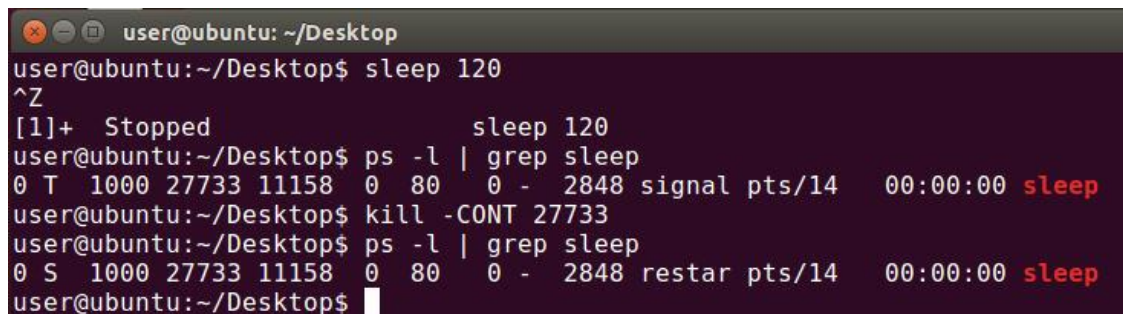
■ 작업 제어 시그널

시그널 명	시그널 번호	의미
SIGCHLD	17	자식 프로세스의 정지 혹은 종료 신호
SIGCONT	18	정지되었던 프로세스를 다시 실행
SIGSTOP	19	실행을 멈춤
SIGTSTP	20	터미널에서의 정지 신호(Control Z)
SIGTTIN	21	백그라운드 프로세스가 읽기 작업을 수행할 때 발생하는 신호, 일시 중지
SIGTTOU	22	백그라운드 프로세스가 쓰기 작업을 수행할 때 발생하는 신호, 일시 중지

- SIGCHLD 시그널은 자식 프로세스가 수행을 멈추거나 종료 시 부모에게 보내는 시그널로, 부모 프로세스에서는 이 시그널을 받아 자식 프로세스를 완전하게 종료하게 만든다. 이에 관련된 함수가 `wait()` 계열의 함수다
- SIGSTOP 시그널은 현재 수행중인 프로세스의 수행을 잠시 정지시킬 때 발생하는 시그널로 프로세스에서 무시하거나 다른 핸들러를 등록할 수 없다. 이 시그널은 SIGCONT 시그널에 의해 실행이 재개된다.

시그널이란

- SIGTSTP 시그널은 사용자가 키보드에서 Control-Z 을 누른 경우 발생하는 시그널로 이 시그널을 수신한 프로세스는 수행을 멈추면서 백그라운드모드로 전환된다. 이 상태를 ‘서스팬드 되었다’ 라고 말한다.
- 다음은 SIGTSTP/SIGCONT 시그널을 이용한 예다.



```
user@ubuntu: ~/Desktop
user@ubuntu:~/Desktop$ sleep 120
^Z
[1]+  Stopped                  sleep 120
user@ubuntu:~/Desktop$ ps -l | grep sleep
0 T  1000 27733 11158  0  80   0 - 2848 signal pts/14   00:00:00 sleep
user@ubuntu:~/Desktop$ kill -CONT 27733
user@ubuntu:~/Desktop$ ps -l | grep sleep
0 S  1000 27733 11158  0  80   0 - 2848 restar pts/14   00:00:00 sleep
user@ubuntu:~/Desktop$
```

- 그림은 sleep 수행 도중 Control Z을 눌러 서스팬드 시켰다가 kill 명령을 이용하여 SIGCONT 시그널을 전송하여 백그라운드 상태에서 다시 수행을 진행하도록 제어하는 화면이다. 시그널이 발생한 후 프로세스의 상태를 확인하면 T (STOP) 상태에서 S(SLEEP) 상태로 전환된 것을 확인할 수 있다.
- 이 밖에도 리눅스에서 지원되는 시그널들을 정리해보면 다음과 같다.

시그널이란

■ 에러 탐지 관련 시그널

시그널 명	시그널 번호	의미
SIGBUS	7	정의되지 않은 메모리로의 접근
SIGFPE	8	부동 소수점 연산 오류
SIGILL	9	허용되지 않는 명령 수행
SIGPIPE	13	읽기 대상이 없는 파이프로 쓰기
SIGSEGV	11	유효하지 않은 메모리로 접근
SIGSYS	31	잘못된 시스템 콜
SIGXCPU	24	CPU 제한시간 초과
SIGXFSZ	25	파일크기 제한 초과

- 주로 프로세스 실행 시 하드웨어 및 소프트웨어의 오류로 발생하는 시그널로 이 시그널을 받은 프로세스는 대부분 종료하도록 설정되어 있다.

시그널이란

■ 타이머 관련 시그널

시그널 명	시그널 번호	의미
SIGALRM	14	알람 시계 완료
SIGVTALRM	26	가상 타이머(프로세스 실행시간 기반 타이머) 완료
SIGPROF	27	프로파일링 타이머 완료

타이머 관련 시그널은 응용 프로그램에서 주로 타이머를 설정하여 특정 시간에 수행할 작업을 예약하는 용도로 사용된다.

■ 기타

시그널 명	시그널 번호	의미
SIGTRAP	5	추적 또는 브레이크 포인트의 트랩
SIGURG	23	소켓 Out-of-Band 데이터가 유효함을 알림

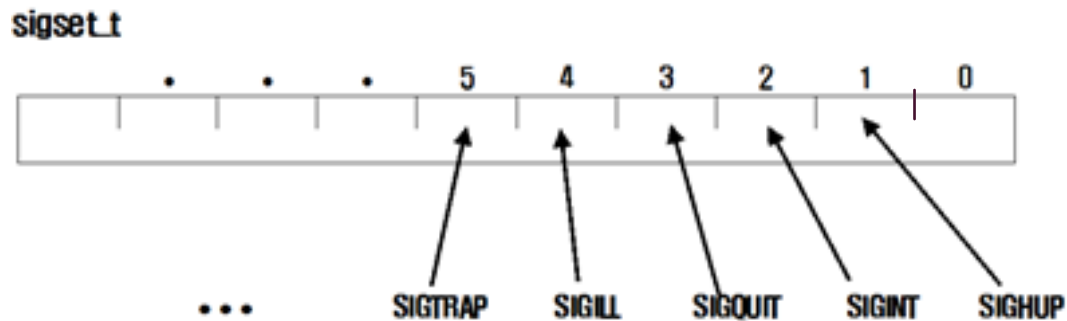
- 지금까지 살펴본 바와 같이 리눅스에는 다양한 종류의 시그널이 존재하며 각 시그널마다 특정 이벤트와 관련이 있다. 일부 시그널에 대해서는 핸들러를 설정하거나 처리 방법을 예약함으로써 프로세스 수행을 제어할 수 있다.

시그널 핸들러

1. 시그널 셋 관리
2. 시그널 핸들러 등록

시그널 셋 관리

- 앞서 살펴보았듯이 시그널의 종류는 아주 다양하다. 한 프로세스에서 이 많은 시그널을 처리한다는 것은 만만치 않은 일이다. 그래서 시그널을 모아서 처리할 수 있도록(시그널 집합) `sigset_t` 자료형이 준비되어 있다.
- 이 자료형은 비트 하나하나가 각 시그널을 의미하도록 구성한 것으로 그림으로 표현하면 다음과 같다.



`sigset_t` 자료형

시그널 셋 관리

- 이 `sigset_t` 자료형의 시그널 집합을 좀더 쉽게 사용할 수 있도록 시그널을 등록, 삭제 할 수 있는 함수가 제공된다

```
#include <signal.h>
```

```
int sigaddset(sigset_t *set, int signo);  
int sigemptyset(sigset_t *set);  
int sigfillset(sigset_t *set);  
int sigdelset(sigset_t *set, int signo);  
int sigismember(sigset_t *set, int signo);
```

함수	의미
<code>sigaddset()</code>	시그널 집합에 특정 시그널 등록
<code>sigemptyset()</code>	시그널 집합 모두 0으로 초기화
<code>sigfillset()</code>	시그널 집합 모두 1로 설정
<code>sigdelset()</code>	시그널 집합으로부터 특정 시그널 삭제
<code>sigismember()</code>	특정 시그널이 시그널 집합에 등록되어있는지 확인

- 이 함수들을 이용하여 프로세스 혹은 스레드에서 블록될 시그널 셋을 구성 및 조정할 수 있으며 이를 시그널 마스크라고 부른다.

ex01.c 를 통해 시그널셋 함수의 의미를 살펴봅시다.

```
#include <signal.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    sigset_t set;

    sigfillset(&set);
    sigdelset(&set, SIGINT);
    sigprocmask(SIG_SETMASK, &set, NULL);

    while(1);
    return 0;
}
```

시그널 핸들러 등록

■ 시그널 핸들러의 종류

- 프로세스에서 시그널을 받았을 때 기본적으로 처리하는 방법은 디폴트 핸들러에 기술되어 있다.
- 그런데 때로는 디폴트로 제공되는 기능이 아닌 사용자가 정의한 작업을 처리해야 하는 경우가 있다. 이런 경우 `signal()`, `sigaction()` 함수 등을 이용하여 시그널 핸들러를 다시 설정할 수 있다. 이 핸들러는 해당 프로세스에서만 유효하며 디폴트 핸들러에는 영향을 주지 않는다. 시그널 중 `SIGKILL`, `SIGSTOP` 시그널에 대해서는 시그널 핸들러를 다시 설정할 수 없다.
- 시그널 핸들러에는 다음과 같은 세 종류의 핸들러를 설정할 수 있다.

핸들러	의미
함수명	사용자가 정의한 핸들러 함수
<code>SIG_DFL</code>	디폴트 핸들러
<code>SIG_IGN</code>	무시

시그널 핸들러 등록

■ 시그널 핸들러 등록 함수

- 시그널 핸들러를 등록할 수 있는 함수에는 `signal()` 과 `sigaction()` 함수가 있다. 이 중 `signal()` 함수는 구형 함수이지만 이전에 작성된 소스 코드를 보면 아직 사용되고 있는 함수이니 우선 가볍게 이 함수를 살펴보고 `sigaction()` 함수를 자세히 살펴보기로 하자.

- **`signal()` 함수**

프로세스에서는 `signal()` 함수를 이용하여 시그널에 대한 처리, 즉 핸들러를 변경할 수 있다. 이 함수의 원형은 다음과 같다.

```
#include <signal.h>
```

```
void (*signal(int sig, void (*func)(int)))(int);
```

- 여기에서 첫 번째 인수에는 시그널 번호를, 두 번째 인수에는 그 시그널에 대한 핸들러 함수를 등록한다. 예를 들면 다음과 같다.

시그널 핸들러 등록

`signal(SIGINT, int_handler);`

`signal(SIGINT, SIG_IGN);`

`signal(SIGINT, SIG_DFL)`

SIGINT (Control C) 신호가 발생할 때마다 다음과 같은 시그널 핸들러가 호출되도록 ex02.c 를 작성해보자.

```
void handler(int signo)
{
    write(1, "Control_C\n", 10);
}
```

시그널 핸들러 등록

■ sigaction() 함수

- sigaction() 함수는 앞서 소개된 signal() 함수와 같이 시그널 핸들러를 등록하는 함수로 보다 안전한 시그널 핸들러 처리를 위해 설계된 함수로 함수의 원형은 다음과 같다.

```
#include <signal.h>

int sigaction(int sig, const struct sigaction *act, struct sigaction *oact);

struct sigaction {
    void (*sa_handler)(int);           // 핸들러 함수, SIG_IGN, SIG_DFL
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask;                 // sa_handler에서 블록킹할 시그널 집합
    int sa_flags;                     // 시그널 처리 동작 변경 플래그
    void (*sa_restorer)(void);        // 현재는 사용되지 않음
};
```

- sigaction() 함수는 첫번째 인자에는 시그널 번호를, 두 번째 인자에는 그 시그널에 대한 새로운 핸들러를, 세 번째 인자에는 이전에 사용하던 시그널 핸들러에 대한 정보를 읽어오도록 변수의 주소를 전달한다.
- 두 번째, 세 번째 인자의 자료형인 struct sigaction 중 sa_mask 멤버에는 시그널 핸들러를 처리하는 동안 블록킹할 시그널의 셋을 구성하여 시그널 마스크 값을 설정한다. 따라서 시그널 핸들러를 안전하게 수행할 수 있다.

시그널 핸들러 등록

- `sa_flags` 멤버에는 시그널을 처리하는 프로세스의 동작을 조정하는 플래그를 설정할 수 있는데, 여기에는 다음과 같은 값들을 표현할 수 있다.

플래그 종류	의미
SA_RESETHAND	시그널을 수신하면 핸들러를 디폴트 핸들러로 재설정, 즉 1회성 핸들러로 등록
SA_RESTART	시그널 핸들러에 의해 중지된 시스템 콜 함수를 자동으로 다시 시작하도록 설정, 이 플래그가 해제되면 시그널 핸들러 수행 후 시스템 콜 함수를 EINTR 에러로 처리
SA_NODEFER	시그널 핸들러가 같은 신호의 중복 수신 허용
SA_NOCLDSTOP	자식 프로세스가 정지하거나 다시 재개할 때 부모 프로세스에게 SIGCHLD 시그널을 전송하지 않음
SA_SIGINFO	리얼타임 시그널에서 시그널의 추가 정보를 저장하는 용도로 사용

- `sigaction()` 함수를 이용하여 특정 시그널에 대한 핸들러와 함께 그 핸들러를 수행하는 동안 블록킹할 시그널 마스크를 만들어 함께 등록하면, 시그널이 연속해서 발생되더라도 핸들러를 수행하는 동안 수신한 시그널을 보류하게 되므로 핸들러의 안전한 처리를 보장할 수 있다. 단, SIGKILL 과 SIGSTOP 시그널은 블록킹 되지 않는다.



ex02.c 에서 작성했던 `signal()` 함수를 `sigaction()` 함수로 수정해보자 (ex03.c)

ex04.c는 시그널 핸들러가 수행되는 동안 시그널이 블록되는 프로그램이다. 프로그램을 실행하여 핸들러를 수행하는 동안 시그널이 블록되는지 확인해보자

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void int_handler( int signo)
{
    int i;

    printf( "int_handler() start.\nPlease waiting for 5 secs!!\n");
    for (i=0; i<5; i++){
        printf( "%d\n", i+1);
        sleep(1);
    }
    printf("int_handler() end\n");
}

int main( void)
{
    struct sigaction act;

    act.sa_handler = int_handler;
    sigfillset( &act.sa_mask);
    sigaction( SIGINT, &act, NULL);

    while(1) {
        printf("sigaction() : signal mask test\n");
        sleep( 1);
    }
}
```

시그널 전송

1. 시그널 전송함수 – kill()
2. 기타함수

시그널 전송 함수 – KILL()

- kill() 함수는 특정 프로세스에게 시그널을 전송할 때 사용하는 함수로 원형은 다음과 같다.

```
#include <sys/types.h>
#include <signal.h>

int kill(pid_t pid, int sig);
```

- 이 함수는 pid로 지정된 프로세스에게 시그널번호 sig 를 전송한다. 성공한 경우에는 0을, 실패한 경우에는 -1 이 전달된다. 실패한 경우 errno 변수에는 다음과 같은 값이 설정된다.

EINVAL	신호가 유효하지 않은 경우
EPERM	프로세스에 대한 권한이 없는 경우
ESRCH	해당 프로세스가 존재하지 않는 경우

ex05.c 를 실행하여 kill() 함수의 의미를 확인해보자

```
#include <signal.h>
#include <stdio.h>
#include <errno.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    if(argc<2){
        fprintf(stderr, "Usage : ex05 pid\n");
        exit(1);
    }
    if(kill(atoi(argv[1]), SIGTERM)==-1){
        if(errno==EINVAL)
            fprintf(stderr, "Invalid Signal \n");
        else if(errno==EPERM)
            fprintf(stderr, "No Permission\n");
        else if(errno==ESRCH)
            fprintf(stderr, "no search process\n");
    }
    return 0;
}
```

기타함수

▪ abort()

- abort() 함수는 시그널 핸들러에서 비정상적으로 종료되어야 하는 경우 이용하는 함수로 이 함수가 호출되면 열려있던 모든 스트림들이 닫히게 된다.

```
#include <stdlib.h>
```

```
void abort(void);
```

▪ raise()

- raise() 함수는 현재 수행중인 프로세스에 시그널을 전송하는 함수로 그 원형은 다음과 같다.

```
#include <signal.h>
```

```
int raise(int sig);
```

〈실습〉

- 리눅스의 프로세스는 종료할 때 부모 프로세스에게 시그널 SIGCHLD를 보내고, 부모 프로세스는 이 시그널을 받아 자식 프로세스의 종료를 처리해야 자식 프로세스가 좀비 프로세스로 남아있지 않게 된다. 다음 조건의 시그널 핸들러를 통해 이 부분을 구현해 보자.
(exercise.c)

〈조건〉

시그널 핸들러 함수 : `child_handler()`

시그널 종류 : SIGCHLD



리눅스 시스템 프로그래밍

6.2장 시그널 응용



목차

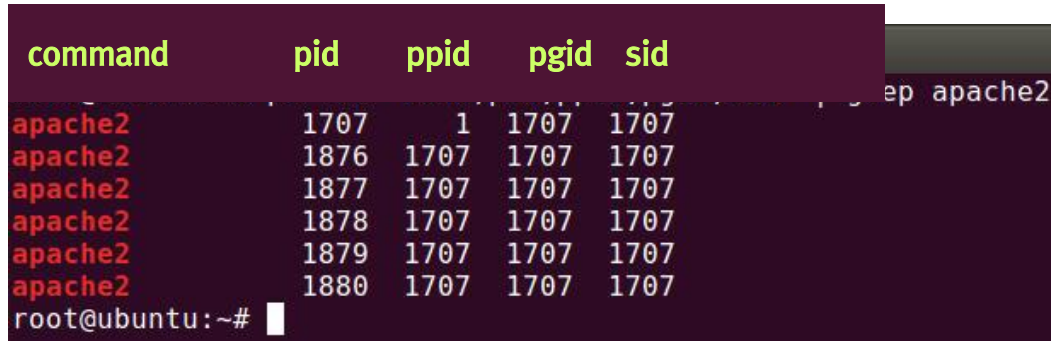
1. 세션과 프로세스 그룹, 그리고 시그널
2. 시그널을 이용한 프로세스 제어
3. 시그널과 타이머

세션과 프로세스 그룹, 그리고 시그널

1. 프로세스 그룹과 세션
2. 시그널 제어

프로세스 그룹과 세션

- 프로세스 그룹이란 `fork()` 함수를 통해 생성되는 일련의 자식 프로세스를 관리하기 위한 그룹으로, 하나의 목적을 위해 생성된 프로세스들의 모임을 의미한다.
- 다음 그림은 우분투 리눅스에서 apache 웹서버 프로세스의 상태를 확인한 화면이다



command	pid	ppid	pgid	sid
apache2	1707	1	1707	1707
apache2	1876	1707	1707	1707
apache2	1877	1707	1707	1707
apache2	1878	1707	1707	1707
apache2	1879	1707	1707	1707
apache2	1880	1707	1707	1707

root@ubuntu:~#

apache 웹 서버 프로세스의 PGID

- 그림에서 각 apache 웹 서버 프로세스는 PID 1707 번의 부모 프로세스로부터 fork 되었고 이 프로세스들은 같은 프로세스 그룹으로 구성되어 있음을 알 수 있다. 같은 프로세스 그룹에 속한 프로세스간에는 파이프(pip)를 이용한 통신이 가능하고 시그널에 대한 처리도 기본적으로 동일한 동작을 수행하게 된다. PID와 PGID 가 같은 프로세스를 프로세스 그룹의 리더라고 부른다. 프로세스 그룹의 리더는 같은 그룹의 프로세스에게 시그널을 전송할 수 있는 기능을 가진다.

프로세스 그룹과 세션

- 세션이란 사용자와 통신을 할 수 있는 일련의 프로세스 그룹들이 수행되는 가상의 공간을 의미하는 말로 일반적으로 로그인이나 터미널 새로 열기 등이 이에 속한다. 이외에 자기 고유의 세션을 관리하는 대몬 프로세스도 새로운 세션을 구성할 수 있다.
- 다음은 프로세스의 세션 ID(SID)를 확인한 화면이다.

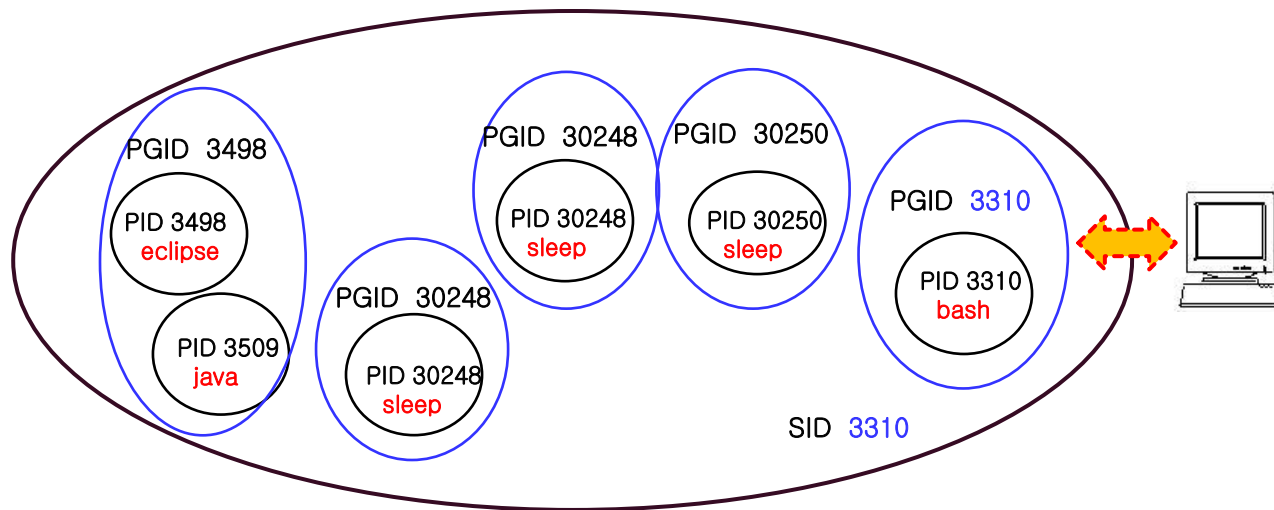
```
user@ubuntu: ~/Desktop
user@ubuntu:~/Desktop$ sleep 300 &
[2] 30248
user@ubuntu:~/Desktop$ sleep 200 &
[3] 30250
user@ubuntu:~/Desktop$

root@ubuntu: ~
root@ubuntu:~# ps -eo "comm,pid,ppid,pgid,sid" | grep sleep
sleep      30248   3310 30248   3310
sleep      30250   3310 30250   3310
root@ubuntu:~# ps -eo "user,command,pid,ppid,pgid,sid" | grep 3310
user      bash      3310   3301   3310   3310
user      /usr/lib/eclipse/eclipse 3498   3310   3498   3310
user      /usr/bin/java -Xms40m -Xmx3 3509   3498   3498   3310
user      /home/user/online_lsp/ch10/ 11785  3509  11785  3310
user      sleep 300  30248  3310  30248  3310
user      sleep 200  30250  3310  30250  3310
root      grep --color=auto 3310  30270 29487 30269 11158
root@ubuntu:~#
```

프로세스 그룹과 SID

프로세스 그룹과 세션

- 앞서 그림에서 같은 SID로부터 생성된 프로세스 그룹을 확인한 화면으로 동일한 SID 3310로부터 3310, 3498, 11785, 30248, 30250의 프로세스 그룹이 생성되었음을 확인할 수 있다. 이때 SID와 PID가 동일한 프로세스를 세션의 리더라고 한다. 이 그림에서는 bash 프로세스가 세션의 리더임을 확인할 수 있다. 그리고 eclipse과 java는 같은 프로세스 그룹으로 이루어져 있으며 나머지 프로세스들은 각각 다른 프로세스 그룹을 구성하고 있음을 알 수 있다. 이를 그림으로 표현하면 다음과 같다.



세션, 프로세스 그룹과 프로세스

프로세스 그룹과 세션

- 이와 같이 특정 목적을 위한 일련의 프로세스, 즉 프로세스 그룹이나 새로운 세션으로 구성하는 방법은 무엇일까? 이제 그 방법을 살펴보자.

- **getpid()/setpgid() 함수**

- 프로세스 그룹을 확인 또는 변경하는 함수로 함수의 원형은 다음과 같다,

```
#include <unistd.h>
```

```
int getpid(pid_t pid);  
int setpgid(pid_t pid, pid_t pgid);
```

- getpid 함수는 인자로 pid 를 전달하면 그 프로세스가 속한 프로세스 그룹의 PGID를 반환한다. Pid를 0으로 전달하면 현재 프로세스의 PID를 의미한다.
- setpgid() 함수의 첫번째 인자인 pid 에는 프로세스의 그룹을 변경하려는 프로세스의 PID를, 두번째 인자인 pgid에는 소속하려는 프로세스 그룹의 pgid를 설정하는 것으로 0으로 설정하면 현재 프로세스의 PID 를 의미한다. 따라서 setpgid(0, 0)은 자기 자신을 새로운 프로세스 그룹의 리더로 설정하는 방법이다.

프로세스 그룹과 세션

▪ setsid() 함수

- 새로운 세션을 구성할 때 이용하는 함수로 원형은 다음과 같다.

```
#include <unistd.h>
pid_t getsid(pid_t pid);
pid_t setsid(void);
```

- getsid() 함수는 인자로 전달되는 pid 의 SID 를 반환하는 함수다.
- setsid() 함수는 새로운 세션을 생성할 때 호출하는 함수로 이 함수를 성공하면 해당 프로세스가 세션의 리더가 되면서 새로운 프로세스 그룹을 자동으로 생성한다.

프로세스 그룹과 시그널 처리

- 프로세스 그룹을 생성하는 방법을 살펴보았으니 이번에는 프로세스 그룹과 시그널의 관계를 살펴보자.
- 프로세스 그룹은 시그널을 전파할 수 있는 기능이 있어 같은 프로세스 그룹 안에 있는 프로세스들을 시그널로 재 시작, 종료, 정지등 제어 할 수 있다. 프로세스 그룹에 대한 시그널 전송은 다음과 같이 kill 명령이나 kill() 함수를 이용하여 수행할 수 있다.

```
kill [-시그널번호] -PGID 또는 kill(-PGID 시그널번호)
```

- 여기서 PGID 에는 반드시 -(음수) 부호를 붙여야 한다. -를 붙이지 않는 경우에는 개별 PID로 간주한다.
- 이제 프로그램을 통해 프로세스 그룹과 시그널의 관계를 이해해보자.

ex01.c 프로그램을 분석하고 실행해보자 실행 결과가 이해되었다면 , 다음 kill() 함수의 수행을 바꾸어 수행해해보고 그 결과를 이해해보자.

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

void usr1_handler(int signo)
{
    printf("pid=%d received SIGUSR1, exiting.\n", getpid());
    exit(EXIT_SUCCESS);
}

void processInfo(char *msg)
{
    printf("%s: my pid = %d, ppid = %d, pgid = %d, sid = %d\n",
        msg, getpid(), getppid(), getpgid(0), getsid(0));
}
```

```

int main(void)
{
    struct sigaction act;

    act.sa_handler=usr1_handler;
    sigfillset(&act.sa_mask);
    act.sa_flags=SA_RESTART;
    sigaction(SIGUSR1, &act, NULL);
    processInfo("main");
    pid_t cpid1, cpid2, cpid3;
    switch(cpid1=fork()) {
    case -1:
        perror("fork");
        break;
    case 0:
        processInfo("child1");
        sleep(5);
        printf("child1 exiting\n");
        exit(EXIT_SUCCESS);
        break;
    default:
        switch(cpid2=fork()) {
        case -1:
            perror("fork2");
            break;
        case 0:
            if(setpgid(0, 0) == -1)
                perror("setpgid error:");
            processInfo("child2");
            switch(cpid3=fork()) {

```

```

                case -1:
                    perror("fork3");
                    break;
                case 0:
                    processInfo("child3");
                    sleep(3);
                    kill(getpid(), SIGUSR1);
                    //kill(-getpgid(getpid()), SIGUSR1);
                    printf("child3 exiting\n");
                    exit(EXIT_SUCCESS);
                    break;
                default:
                    wait(NULL);
                    printf("child2 exiting\n");
                    exit(EXIT_SUCCESS);
                    break;
            }
        }
    }
    sleep(2);
    //kill(0, SIGUSR1);
    waitpid(cpid2, NULL, 0);
    waitpid(cpid1, NULL, 0);
    printf("main exiting\n");
    return 0;
}

```

시그널을 이용한 프로세스 제어

1. 시그널 마스크 관리
2. 지연된 시그널 처리

시그널 마스크 관리

■ 시그널 셋 관리 함수

- 앞서 학습했던 시그널 핸들러 과정에서 시그널 셋 관리함수를 살펴본 바 있다. 다시 한번 리뷰해 보자. 시그널의 종류는 60여가지가 넘기 때문에 개별적으로 다루기 보다는 집합(set)으로 묶어서 다루며 이에 관련된 함수는 다음과 같다,

```
#include <signal.h>
```

시그널 셋 관리 함수	의미
int sigaddset(sigset_t *set, int signo);	시그널 집합에 특정 시그널 등록하는 함수
int sigemptyset(sigset_t *set);	시그널 집합 모두 0으로 초기화하는 함수
int sigfillset(sigset_t *set);	시그널 집합 모두 1로 설정하는 함수
int sigdelset(sigset_t *set, int signo);	시그널 집합으로부터 특정 시그널 삭제하는 함수
int sigismember(sigset_t *set, int signo);	특정 시그널이 시그널 집합에 등록되어있는지 확인하는 함수

- 이 함수들을 이용하여 프로세스 혹은 스레드에서 블록될 시그널 셋을 구성 및 조정할 수 있으며 이를 시그널 마스크라고 부른다.

시그널 마스크 관리

■ 시그널 마스크 등록

- 시그널 마스크란 프로그램이 수행되는 동안 블로킹 할 시그널 셋(set)을 의미하는 말로, 프로세스나 스레드의 시그널 마스크를 등록하는 함수는 다음과 같다.

```
#include <signal.h>
```

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oset);  
int pthread_sigmask(int how, const sigset_t *set, sigset_t *oset);
```

- 여기서 sigprocmask() 함수는 프로세스에, pthread_sigmask() 함수는 스레드에 적용하는 함수로 원형은 동일하다. 다만 sigprocmask 함수는 반환 값이 성공 여부를 의미하고 에러 코드는 errno 변수에 설정되는 반면, pthread_sigmask 함수의 경우 에러 코드가 반환된다.
- 첫번째 인자인 how의 설정에 따라 프로세스의 시그널 마스크의 동작 방식을 설정할 수 있다. 두번째 인자인 set에는 새로운 시그널 마스크 값을 등록하고, 세 번째 인수 oset에는 현재의 시그널 마스크를 받아올 수 있다. 여기에서 how 에 표현할 수 있는 값은 다음과 같다.

시그널 마스크 관리

- 시그널 마스크 등록

SIG_BLOCK	set의 시그널들이 시그널 마스크에 추가된다.
SIG_SETMASK	시그널 마스크 값을 set으로 설정한다.
SIG_UNBLOCK	set의 시그널을 시그널 마스크로부터 삭제한다.

- 만약 set이 NULL이라면 how는 이용되지 않는다. 단지 현재의 시그널 마스크 값을 읽어올 뿐이다.



ex02.c 를 실행하여 sigprocmask() 함수에 대해 이해해보자

지연된 시그널 처리

■ sigpending() 함수

- 프로세스 수행 중 시그널 마스크에 포함된 시그널이 도착되면 그 시그널은 보류 상태로 남아 있게 된다. 다음 함수를 이용하면 보류된 시그널들을 확인할 수 있다.

```
#include <signal.h>
```

```
int sigpending(sigset_t *set);
```

- 이 함수를 호출하면 현재 보류되어 있는 시그널들을 set으로 전달해준다. 성공하면 0을 실패하면 -1을 반환한다.

ex03.c 를 통해 sigpending() 함수를 이해해보자

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

int main( void)
{
    sigset_t sigset;
    sigset_t pendingset;
    int i = 0;

    // 모든 시그널을 블록
    sigfillset( &sigset);
    sigprocmask( SIG_SETMASK, &sigset, NULL);

    printf("waiting for signal...(10 sec.)\n");
    for(i=0; i<10; i++) {
        printf("카운트: %d\n", i);
        sleep( 1);
    }

    if(sigpending(&pendingset)==0) {
        printf("I got : \n");
        if (sigismember( &pendingset, SIGINT))
            printf( "SIGINT\n");
        if (sigismember( &pendingset, SIGQUIT))
            printf( "SIGQUIT\n");
        if (sigismember( &pendingset, SIGUSR1))
            printf( "SIGUSR1.\n");
        if (sigismember( &pendingset, SIGUSR2))
            printf( "SIGUSR2.\n");
    }
    return 0;
}
```

지연된 시그널 처리

■ sigsuspend() 함수

- sigsuspend() 는 시그널 집합에 포함되어있는 시그널이 프로세스에 도착할 때까지 수행을 멈추는 함수다. 그 원형은 다음과 같다.

```
#include <signal.h>
```

```
int sigsuspend(const sigset_t *sigmask);
```

- 이 함수를 호출하면 프로세스의 시그널 마스크가 sigmask 로 바뀌면서 프로세스의 실행을 일시적으로 중단한다. 시그널이 도착하면 그 시그널의 핸들러를 수행한 후 프로세스의 실행이 재개된다.
- 만약 시그널 핸들러에서 프로그램이 종료되면 sigsuspend() 함수는 리턴되는 값이 없고, 종료되지 않으면 -1 값이 반환된다. 이때 errno 변수에는 EINTR가 설정된다.



모든 시그널을 블록시키고 SIGINT 신호가 들어올 때까지 기다리는 ex04.c 를 작성해보자.

시그널과 타이머

1. 알람 타이머
2. 인터벌 타이머

알람 타이머

- alarm() 함수는 특정 시간 후에 SIGALRM 시그널을 발생시키는 함수로 일정 시간 후에 처리해야 할 일이 있을 때 유용하게 사용할 수 있다. 함수의 원형은 다음과 같다.

```
#include <unistd.h>
```

```
unsigned int alarm(unsigned int seconds);
```

- alarm() 함수는 인수로 넘겨주는 시간(초 단위) 후에 SIGALRM 시그널을 발생시켜준다. 인수에 0을 넘겨주면 알람의 요청을 취소하게 된다.

ex05.c를 이용하여 alarm() 함수와 SIGALRM 시그널에 대한 처리 과정을 알아보자.

```
#include <stdio.h>
#include <sys/types.h>
#include <signal.h>
#include <unistd.h>

static int alarm_flag=0;

void alarm_handler(int sig)
{
    alarm_flag=1;
}

int main(void)
{
    struct sigaction act;

    act.sa_handler=alarm_handler;
    sigfillset(&act.sa_mask);
    act.sa_flags=SA_RESTART;
    sigaction(SIGALRM, &act, NULL);
    alarm(5);
    pause(); // wating for any signal
    if(alarm_flag)
        printf("ALARM\n");
    return 0;
}
```

인터벌 타이머

- 앞서 살펴본 alarm 함수는 한번 리셋되면 다시 설정해야 한다. 따라서 인터벌 타이머로 사용하기에는 적절하지 않다. 인터벌 타이머 설정은 getitimer()/ setitimer() 함수를 이용하는 것이 일반적이다. 이제 이 함수에 대하여 알아보도록 하자.
- 이 두 함수의 원형은 다음과 같다.

```
#include <sys/time.h>

int getitimer(int which, struct itimerval *val);
int setitimer(int which, const struct itimerval *val,
              struct itimerval *oval);
```

- getitimer()함수는 현재 설정된 timer 값을 확인할 때, setitimer() 함수는 타이머를 새로 설정할 때 사용한다. 여기에서 첫 번째 인자인 which에는 다음과 같은 타이머의 종류를 설정한다.

타이머 종류	의미
ITIMER_REAL	실시간 타이머로, 만료 후 SIGALRM 시그널 발생
ITIMER_VIRTUAL	프로세스의 가상시간 타이머로, 만료후 SIGVTALRM 시그널 발생
ITIMER_PROF	프로세스의 가상 시간 및 프로세스 실행 시간에 기반한 타이머로, 만료후 SIG_PROF 시그널 발생, 주로 CPU 소모 시간을 측정하는 프로파일링 측정 시 이용

인터벌 타이머

- 두번째 인자에는 현재 타이머 설정 값을 읽어오거나 새로 설정할 타이머를 전달한다. 세번째 인자에는 새로운 타이머 설정 시 이전에 사용하던 타이머를 받아올 수 있다. 여기서 사용된 구조체 `struct itimerval` 은 `time.h` 헤더 파일에 다음과 같이 선언되어 있다.

```
struct itimerval {
    struct timeval it_interval;
    struct timeval it_value;
};

struct timeval {
    time_t tv_sec;
    time_t tv_usec;
};
```

- 여기서 `it_value` 멤버에는 현재 시간 간격을, `it_interval` 멤버에는 타이머 만료시 재설정할 시간을 설정한다. 한번만 설정하면 자동으로 재설정되므로 `alarm()` 함수처럼 사용자가 매번 재설정할 필요가 없다.

ex06.c 는 인터벌 타이머에 대한 프로그램이다 소스를 확인한 후 실행하면서 ex05.c 의 수행결과와 비교해보자.

```
...

void interval_handler(int sig)
{
    printf("<interval handler> called : %d\n", time(NULL));
}

int main(void)
{
    struct sigaction act;
    struct itimerval itimer;

    act.sa_handler=interval_handler;
    sigfillset(&act.sa_mask);
    act.sa_flags=SA_RESTART;
    sigaction(SIGALRM, &act, NULL);

    printf("<main> Current Time : %d\n", time(NULL));
    memset(&itimer, 0, sizeof(itimer));
    itimer.it_value.tv_sec=5;
    itimer.it_interval.tv_sec=2;
    setitimer(ITIMER_REAL, &itimer, NULL);

    while(1)
        pause(); // wating for any signal
    return 0;
}
```

〈실습〉

- exercise.c 프로그램은 인터럽트 신호가 들어올 때까지 USR1과 USR2 시그널이 몇 개 발생했는지 계산하는 프로그램이다. 다른 시그널의 간섭을 통제하기 위해 프로세스에 다른모든 시그널을 블록킹하도록 설정해보자.